Multithreading

1. Conceito Fundamental

Multithreading é a capacidade de um sistema operacional ou aplicação de gerenciar múltiplos fluxos de execução (*threads*) concorrentemente ou em paralelo dentro do espaço de endereçamento de um único processo.

2. Processos vs. Threads

- **Processos:** Possuem espaços de endereçamento **isolados** de memória. A comunicação exige mecanismos de Comunicação Interprocessos (*IPC*) e a troca de contexto é custosa, pois exige o remapeamento da tabela de páginas de memória.
- **Threads (Processos Leves):** Linhas de execução pertencentes a um mesmo processo.
  - **O que compartilham:** O mesmo espaço de endereçamento (seção de código e dados/variáveis globais), recursos do SO (arquivos abertos, sinalizadores) e a memória heap.
  - **O que possuem individualmente:** Contador de Programa (*PC*), conjunto de registradores da CPU e Pilha de Execução (*Stack*) própria.
  - **Vantagem:** A criação e a troca de contexto entre threads são significativamente **menos custosas** em comparação com processos.

3. Modelos de Mapeamento de Threads

Existem três arquiteturas fundamentais para conectar threads de usuário às threads gerenciadas pelo Kernel do SO:

1. **Muitos-para-Um (**$N:1$ **- *User-Level Threads / ULT*):**
   - Múltiplas threads de usuário são gerenciadas por uma biblioteca em espaço de usuário e mapeadas para uma única thread do Kernel.
   - *Vantagem:* Altamente eficiente e rápido para trocas de contexto.
   - *Desvantagem:* Se uma thread fizer uma chamada de sistema bloqueante (ex.: leitura de disco), **todo o processo é bloqueado**. Não aproveita múltiplos núcleos de CPU (*multicore*).
2. **Um-para-Um (**$1:1$ **- *Kernel-Level Threads / KLT*):**
   - Cada thread de usuário é diretamente mapeada para uma thread do Kernel.
   - *Vantagem:* Se uma thread bloquear, outras continuam executando. Permite execução paralela física real em múltiplos núcleos.
   - *Desvantagem:* Criar uma thread de usuário exige a criação de uma thread no Kernel, aumentando o *overhead*.
3. **Muitos-para-Muitos (**$M:N$ **- Modelo Híbrido):**
   - Multiplexa M threads de usuário em N threads de Kernel ($M \geq N$).
   - *Vantagem:* Combina o melhor dos dois mundos, garantindo alta eficiência e concorrência sem bloquear todo o processo.

4. Vantagens do Multithreading

- **Responsividade:** Permite que aplicações (como interfaces gráficas ou servidores) continuem respondendo a interações enquanto tarefas pesadas rodam em segundo plano.
- **Compartilhamento de Recursos:** Dispensa o uso de estruturas complexas de IPC, pois o acesso a variáveis e dados é direto pelo mesmo espaço de memória.
- **Economia de Recursos:** Criar, manter e alternar threads consome muito menos tempo e memória da CPU do que alocar novos processos.
- **Utilização de Arquiteturas Multicore:** Permite o aproveitamento pleno do paralelismo de hardware, onde diferentes threads do mesmo processo são alocadas em núcleos físicos distintos.

5. Desafios do Multithreading

- **Condições de Corrida (*Race Conditions*):** Como as threads compartilham a mesma memória global, acessos simultâneos sem sincronização onde ocorra pelo menos uma operação de escrita causam inconsistência de dados.
- **Sincronização Obrigatória:** Exige o emprego de primitivas de sincronização — como **Mutexes**, **Semáforos** e **Monitores** — para garantir a exclusão mútua em regiões críticas.
- **Riscos de Concorrência:** O mau uso das travas de sincronização pode gerar impasses (**Deadlocks**) ou inanição (**Starvation**).

Problemas Clássicos de Concorrência

- **Livelock:** As threads mudam de estado continuamente em resposta umas às outras, mas nenhuma progride (diferente do *Deadlock*, onde ficam bloqueadas paradas).
- **Inversão de Prioridade:** Uma thread de baixa prioridade segura um recurso que uma thread de alta prioridade precisa, fazendo com que threads de prioridade média passem à frente da mais prioritária.
- **Problemas Clássicos da Literatura:**
  - Produtor-Consumidor (Buffer Limitado)
  - Leitores-Escritores
  - Jantar dos Filósofos

6. Primitivas de Sincronização e Estratégias Alternativas

- **Operações Atômicas e Instruções de Hardware:** Instruções como *Test-and-Set* (TAS) e *Compare-and-Swap* (CAS) executadas no nível do hardware sem interrupção.
- **Spinlocks:** Travas de espera ocupada (*busy-waiting*); gastam ciclos de CPU, mas são úteis quando o tempo de espera esperado é menor que o custo de uma troca de contexto.
- **Estruturas Lock-Free:** Algoritmos concorrentes que não utilizam travas explícitas, evitando deadlocks por meio de operações atômicas.

7. Ciclo de Vida e Estados da Thread

- **Criada (New/Spawned)**
- **Pronta (Ready):** Aguardando alocação da CPU.
- **Executando (Running):** Alocada em um núcleo de processamento.
- **Bloqueada/Esperando (Blocked/Waiting):** Aguardando I/O, sinal ou liberação de trava/mutex.
- **Terminada (Terminated/Zombie):** Execução concluída, aguardando desalocação de sua pilha e identificador.

8. Detalhes de Implementação Prática

- **Thread Local Storage (TLS):** Mecanismo para alocar variáveis que são estáticas/globais dentro do contexto de uma thread específica, sem que outras threads do mesmo processo enxerguem ou modifiquem seu valor.
- **Cancelamento de Threads:** Pode ser **assíncrono** (encerrada imediatamente pelo SO, com alto risco de deixar travas presas ou recursos órfãos) ou **diferido** (*deferred*, onde a thread verifica periodicamente pontos de cancelamento seguros).
- **Padrão Thread Pool:** Em sistemas reais de alto tráfego (servidores web, APIs), criar threads sob demanda para cada requisição satura a memória. Uma *Thread Pool* pré-aloca um número fixo de threads reutilizáveis a partir de uma fila de tarefas.

9. Concorrência vs. Paralelismo (Diferença Conceitual Clássica)

- **Concorrência:** Lidar com múltiplas tarefas ao mesmo tempo através de intercalação/escalonamento no tempo (ocorre mesmo em uma CPU com um único núcleo, via *time slicing*).
- **Paralelismo:** Executar múltiplas tarefas simultaneamente no mesmo instante físico de tempo (exige múltiplos núcleos ou múltiplos processadores).

10. Leis de Desempenho e Escalabilidade

Adicionar threads nem sempre torna o programa indefinidamente mais rápido:

- **Lei de Amdahl:** Demonstra que o ganho máximo de desempenho de um programa ao usar múltiplos processadores é limitado pela fração do código que é estritamente sequencial:

$$S = \frac{1}{(1-p)+\frac{p}{N}}$$

- *(onde p é a proporção paralelizada e N é o número de núcleos).*
- **Lei de Gustafson:** Argumenta que, com mais processadores disponíveis, o tamanho do problema tratado pela aplicação também tende a crescer, aproveitando melhor o poder computacional.

Programação cliente-servidor

1. O que é, em sistemas operacionais, o modelo de programação cliente-servidor?

No contexto de sistemas operacionais (SO), o modelo de programação cliente-servidor é um paradigma de estruturação e comunicação entre processos (IPC - *Inter-Process Communication*) baseado na **troca de mensagens** (*message passing*). Nele, os processos são divididos em dois papéis fundamentais:

- **Cliente:** Um processo que requisita a execução de uma operação ou o acesso a um recurso enviando uma mensagem explícita (requisição).
- **Servidor:** Um processo que aguarda passivamente requisições, processa o pedido ao recebê-lo e, opcionalmente, devolve uma resposta (resultado ou status).

Dentro do SO, esse modelo é a espinha dorsal de arquiteturas baseadas em **micronúcleo** (*microkernel*, como MINIX, QNX ou Mach): a maioria dos serviços do sistema (sistema de arquivos, gerenciador de processos, pilha de rede) roda em espaço de usuário (*user space*) como servidores autônomos. Quando um programa de usuário (cliente) deseja abrir um arquivo, ele envia uma mensagem IPC ao servidor de arquivos através de primitivas como `send()` e `receive()`, em vez de disparar uma chamada de sistema tradicional (*syscall*) para um núcleo monolítico.

RPC e RMI (Remote Procedure Call): Mencionar que, do ponto de vista do programador, a troca bruta de mensagens (`send`/`receive`) costuma ser abstraída por **RPC** (Chamada de Procedimento Remoto). O cliente executa o que parece ser uma chamada de função local comum, mas um *stub* empacota os parâmetros (*marshalling*) e despacha a mensagem IPC ao processo servidor.

Sincronismo vs. Assincronismo: Detalhar a semântica de bloqueio:

- **Síncrono (Bloqueante):** O cliente envia a requisição e suspende sua execução até receber o resultado (comportamento padrão de muitas chamadas de sistema).
- **Assíncrono (Não-bloqueante):** O cliente envia o pedido e continua sua execução, coletando a resposta depois via *polling*, *callbacks*, sinais ou eventos (*event loops*).

2. O que diferencia o modelo de programação cliente-servidor de outros modelos? Dê exemplos de outros modelos de programação concorrente?

A principal distinção reside no **desacoplamento**, na **assimetria de papéis** e no **mecanismo de coordenação**:

1. **Assimetria de papéis:** A interação é estritamente dirigida: o cliente inicia a transação solicitando um serviço; o servidor atende.
2. **Isolamento de espaço de endereçamento:** Não há compartilhamento obrigatório de memória. Cada processo opera em seu próprio espaço virtual protegido, comunicando-se via mensagens.
3. **Localidade transparente:** A semântica da interface (`requisitar -> responder`) é idêntica quer os processos estejam no mesmo núcleo, em núcleos diferentes ou em máquinas distintas na rede.

Exemplos: Memória Compartilhada (*Shared Memory*) com Primitivas de Sincronização; Produtor-Consumidor / *Pipeline* / Streams; Modelo de Atores (*Actor Model*).

Exemplos Adicionais de Modelos Concorrentes:

- **Peer-to-Peer (P2P):** Modelo simétrico onde nós atuam simultaneamente como clientes e servidores (*servents*), sem autoridade central.
- **Publicador-Assinante (Pub/Sub):** Desacoplamento temporal e espacial completo, mediado por tópicos ou corretores de mensagens (*message brokers*), onde emissores não sabem quem são os consumidores.

Acoplamento Temporal: No cliente-servidor tradicional, ambos geralmente precisam estar ativos no momento da transação (a menos que haja uma fila intermediária persistente), ao contrário de filas de mensagens assíncronas puras.

3. O modelo cliente-servidor é amplamente usado em sistemas distribuídos. Este modelo pode também ser usado em um único computador? Por quê?

Sim, e é extensivamente utilizado em sistemas locais.

Por que é possível e vantajoso:

- **Mecanismos de IPC local:** Os sistemas operacionais oferecem canais eficientes de comunicação interna que implementam essa semântica sem passar por placas de rede físicas:
  - *UNIX Domain Sockets* (muito mais rápidos que sockets TCP de loopback, pois evitam o overhead de empacotamento IP/checksums).
  - Filas de mensagens POSIX/System V.
  - Portas/MIPCs em micronúcleos (ex.: *Mach ports* no subsistema XNU do macOS).
  - *Named Pipes* (FIFOs) e ALPC (*Advanced Local Procedure Calls*) no Windows.
- **Modularidade e estabilidade:** Se o servidor de impressão ou o servidor gráfico falhar, ele pode ser reiniciado sem derrubar o kernel ou o restante do sistema operacional.
- **Segurança e princípio do menor privilégio:** O servidor pode rodar com privilégios específicos (ex.: acesso direto ao hardware de vídeo), enquanto os clientes rodam em modo restrito.

Exemplos Práticos e Concretos do Dia a Dia:

- **Servidores de Interface Gráfica / Janelas:** O **X11 (X Window System)** ou **Wayland** no Linux. As aplicações de interface gráfica (Firefox, editores, terminais) são clientes que se conectam ao servidor gráfico local via UNIX Domain Socket para solicitar desenho de janelas e receber eventos de mouse/teclado.
- **Servidores de Áudio: PulseAudio** ou **PipeWire**, onde as aplicações clientes enviam seus fluxos de som para o servidor misturar e enviar ao driver.
- **Daemons de Sistema e Contêineres:** O **Docker Daemon (`dockerd`)**, onde o comando de terminal `docker run ...` atua apenas como cliente CLI enviando requisições REST/gRPC via socket local para o daemon executar as operações reais.
- **D-Bus:** O barramento de mensagens mais comum do Linux para IPC cliente-servidor entre desktop e serviços de sistema (ex.: gerenciador de energia, NetworkManager).

4. Quando um processo aguarda a resposta de outro, isso caracteriza um modelo cliente-servidor?

Não necessariamente. A espera por uma resposta é uma condição comum, mas não suficiente para definir o modelo cliente-servidor.

Para caracterizar o modelo cliente-servidor, deve haver uma **relação funcional assimétrica e estruturada de prestação de serviços**:

- O servidor deve oferecer uma interface bem definida de operações/serviços reutilizáveis.
- O servidor geralmente possui ciclo de vida independente dos clientes (fica em *loop* esperando comandos).
- A relação é de requisição de serviço e retorno de resultado.

Contraexemplos Claros onde há espera, mas NÃO é Cliente-Servidor:

- **Sincronização Pai-Filho:** Um processo pai que invoca wait() ou waitpid() aguardando a finalização de seu processo filho. Há bloqueio e retorno de um status de saída, mas a relação é hierárquica de ciclo de vida, não de consumo de serviços.
- **Barreiras de Sincronização (Barrier Sync):** Múltiplos processos ou threads executam partes de um cálculo paralelo e aguardam todos atingirem uma barreira comum para prosseguir. A relação é estritamente entre pares (*peers*), totalmente simétrica.
- **Bloqueio por Recurso Compartilhado (Mutex / Lock):** Um processo aguardando outro liberar uma seção crítica através de um semáforo ou trava. Nenhum está prestando serviço ao outro; estão apenas competindo por exclusão mútua.
- **Pipeline Simples (Shell Pipes):** Em cat arquivo | grep termo, o comando grep espera blocos de dados de cat, caracterizando um fluxo unidirecional de dados (produtor-consumidor), sem a semântica de "solicitação → resposta".

Programação em sockets

1. O Socket na Arquitetura do Kernel

- Abstração VFS (Virtual File System): O socket é instanciado como um descritor de arquivo (`struct file` no Linux). Ele possui uma entrada na tabela de descritores de arquivos do processo (`task_struct->files`), mas aponta para uma interface interna de rede (`struct socket` conectada a uma `struct sock`), e não para um inode de disco convencional.
- Espaço de Usuário vs. Espaço de Kernel:
  - O processo do usuário lida apenas com um inteiro (*File Descriptor*).
  - O Kernel aloca e gerencia toda a memória subjacente à conexão de forma protegida no *ring 0*.
- Buffers do Kernel (Socket Buffers - `sk_buff`):
  - Para cada socket, o SO aloca dois buffers circulares distintos: Buffer de Recepção (Rx) e Buffer de Transmissão (Tx).
  - `write()` / `send()` não envia dados diretamente para a placa; apenas copia os bytes da memória do processo (espaço de usuário) para o buffer de transmissão do Kernel (`copy_from_user`). O subsistema de rede do SO decide quando e como despachar os pacotes.
  - `read()` / `recv()` apenas copia os bytes do buffer Rx do Kernel para o buffer do usuário (`copy_to_user`).

2. A Mecânica Interna das Chamadas de Sistema (Syscalls)

- `socket()`: Solicita ao subsistema de rede do Kernel a alocação de estruturas de dados de transporte/rede e associa um novo FD na tabela do processo.
- `bind()`: Registra o socket nas tabelas hash internas do Kernel (ex.: tabela de portas locais/bindings), garantindo exclusividade de porta por interface.
- `listen()`: Converte o socket para o modo passivo e inicializa duas filas internas gerenciadas pelo Kernel:

1. Fila SYN (Incompleta): Conexões no meio do *handshake* TCP (SYN recebido, SYN-ACK enviado, aguardando ACK final). Alvo comum de ataques de exaustão de recursos (*SYN Flood* / mitigado por *SYN Cookies*).
2. Fila de Aceite (Accept Queue / Completa): Conexões cujo *three-way handshake* foi totalmente concluído pelo Kernel e estão prontas para serem entregues à aplicação. O parâmetro `backlog` da syscall limita essa capacidade.

- `accept()`: Retira a primeira conexão pronta da fila de aceite e aloca um novo descritor de arquivo (FD) com novas estruturas de estado para essa conexão cliente específica, deixando o FD original livre para monitorar novos pedidos de conexão.

3. IPC Local: UNIX Domain Sockets (`AF_UNIX`) vs. Pilha de Rede (`AF_INET`)

Sob o ponto de vista de SO, essa é uma das distinções mais importantes:

- Zero Overhead de Protocolo: UNIX Domain Sockets dispensam encapsulamento IP, cabeçalhos TCP, cálculo de checksums de integridade e verificação de números de sequência.
- Comunicação por Memória Compartilhada no Kernel: Os dados fluem diretamente de um buffer no Kernel para outro através de cópias de memória simples ou filas circulares protegidas por travas internas.
- Segurança Baseada no Sistema de Arquivos: Como o socket UNIX é representado por um arquivo especial (tipo `s` no `ls -l`), o SO aplica diretamente as permissões de arquivo POSIX (chmod/chown) para controle de acesso, em vez de exigir regras de firewall (*iptables/nftables*).
- Passagem de Descritores de Arquivos: Um recurso único do `AF_UNIX` é a capacidade de um processo passar um FD aberto diretamente para outro processo via `sendmsg()` com dados de controle (`SCM_RIGHTS`), permitindo delegação de recursos no SO.

4. Ciclo de Vida do Processo e Gerenciamento de Bloqueio

- Filas de Espera do Kernel (*Wait Queues*):
  - Quando um processo invoca `recv()` em um socket bloqueante vazio, o escalonador do SO remove o processo do estado Executando (Running), insere-o na *Wait Queue* atrelada àquele socket e coloca o processo em estado Bloqueado (Interruptible Sleep).

○ A chegada de um pacote físico gera uma Interrupção de Hardware (IRQ) → o driver/Kernel processa o pacote em *softirq* →coloca os dados no buffer Rx → acorda o processo da *wait queue*, movendo-o para a fila de Pronto (Ready).

- Mecanismos de Notificação e I/O Multiplexing no SO:

○ Problema C10K: Criar uma thread/processo por cliente esgota a memória do kernel (cada pilha consome memória de kernel) e satura o escalonador com custo de troca de contexto (*context switch*).

○ `select()` / `poll()`: Custo O(N)O(N)O(N) no Kernel. O processo precisa repassar a lista inteira de FDs em toda chamada, forçando o kernel a inspecionar FD por FD linearmente.

○ `epoll` (Linux): Estrutura no Kernel mantida por uma Árvore Rubro-Negra (para FDs monitorados) e uma Lista Encadeada de Prontos (Ready List). Quando um evento de rede ocorre, o driver/Kernel já insere o socket na lista de prontos através de *callbacks*. O processo apenas acorda e obtém os eventos prontos em tempo O(1)O(1)O(1).

5. Herança de Descritores, `fork()` e Encerramento

- Comportamento no `fork()`:

○ Quando um servidor faz `fork()`, a tabela de descritores é clonada. Pai e filho apontam para a mesma entrada na tabela global de arquivos do Kernel (compartilham o mesmo ponteiro e contadores de referência do socket).

○ Se o pai não fechar o socket cliente aceito (`close(client_fd)`), a conexão TCP não será finalizada quando o filho terminar, pois a contagem de referências no Kernel permanece >0> 0>0.

- A Flag `FD_CLOEXEC` (`SOCK_CLOEXEC`): Garante que, se o processo disparar uma chamada `execve()`, o descritor de socket não vaze para o novo programa executado.
- Tratamento de `close()` vs. `shutdown()`:

○ `close()` decrementa o contador de referências do FD e só envia o pacote TCP `FIN` quando a contagem atinge zero.

○ `shutdown()` atua diretamente no protocolo no Kernel, permitindo fechar apenas o canal de escrita ou leitura (encerramento unilateral/half-close), independentemente de quantas referências o FD possua.

3. Starvation e Thundering Herd Problem

- Thundering Herd (Manada Enfurecida): Ocorre quando múltiplos processos/threads estão bloqueados em `accept()` ou monitorando o mesmo socket de escuta em um `epoll`. Quando uma nova conexão chega, o Kernel acorda *todos* os processos; porém, apenas um obtém a conexão e os demais voltam a dormir, gerando desperdício massivo de CPU.
- Mitigações do Kernel:

○ O Kernel Linux moderno já serializa o `accept()` para acordar apenas uma thread.

○ A flag `SO_REUSEPORT` permite que múltiplos sockets/processos façam `bind()` na mesma porta, fazendo com que o próprio Kernel balanceie as conexões entre as filas de forma justa.

4. Backpressure e Bloqueio no Envio (`write`/`send`)

- É comum associar bloqueio apenas à leitura (`recv`), mas a chamada `send()` também bloqueia se o Buffer de Transmissão (Tx) estiver cheio.
- Causa do Kernel: Se o receptor do outro lado da conexão for lento ou a rede estiver congestionada, a janela de fluxo do TCP (*TCP Window Size*) fecha e o Kernel local para de transmitir. Quando o buffer Tx local enche, o processo remetente é colocado em estado *Blocked/Sleep* até que a rede libere espaço no buffer.